Hunting and exploiting bugs in kernel drivers

Andras Kabai

Hacktivity 2012

Who am I

Andras Kabai
OSCP, OSCE, OSEE, GPEN, GWAPT, GREM, GXPN, CEH
Manager of Deloitte Hungary's Security & Privacy group

Previously
- Senior IT Security Specialist / R&D Manager
- CERT-Hungary, National CyberSecurity Center

10+ years in IT Security
Penetration testing, reverse engineering, malware analysis, vulnerability research, exploitation, incident handling

Member of gula.sh team (CyberLympics 2012 World Finals)

contact (_at_) kabaiandras.hu

Introduction

- Exploiting kernel drivers has become increasingly popular

- Exploiting a vulnerability in kernel space may give us the possibility to take control of the entire system

- Thousands of possible target (drivers shipped with OS, 3rd party drivers)

- General goal: privilege escalation

- Focus on Win32 kernel drivers

Ring3 vs Ring0

- Different access levels to resources, usually enforced by the hardware

- User mode (Ring3)
  - No access to hardware, user mode programs has to call the system to interact with the hardware
  - Restricted environment, separated process memory
  - Memory: 0x00000000 to 0x7FFFFFFF
  - Hard to crash the system

- Kernel mode (Ring0)
  - Full access to hardware
  - Unrestricted access to... everything
    - Kernel code, kernel structures, memory, processes, hardware
  - Memory: 0x80000000 to 0xFFFFFFFF
  - Easy to crash the system

Windows Driver Model (WDM) Drivers

- Framework for “Device drivers”
- Introduced with Win98 and Win2000 to replace VxD
- Windows Driver Kit (WDK)
  - Tools
  - Documentation
  - Samples
- Communicates with I/O Request Packet (IRP)

Communication basics

- Driver creates a DRIVER_OBJECT with IoCreateDevice()

- DRIVER_OBJECT holds “Major function” pointers to driver functions
  - IRP_MJ_DEVICE_CONTROL points to a function that is responsible for IOCTL calls

- Driver will be available for user space programs through its symbolic link (e.g. \Device\DEVICENAME)

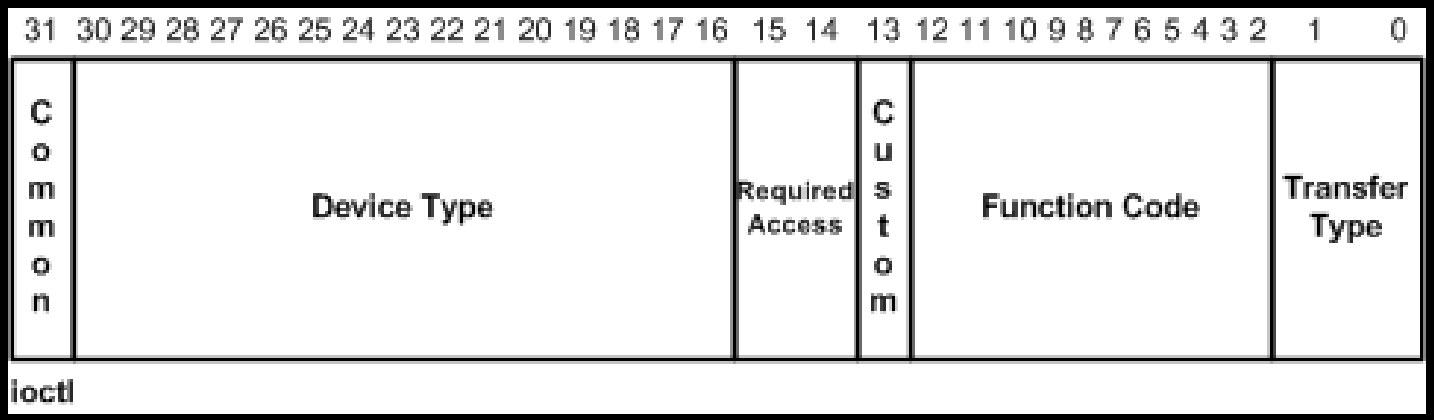
- User space program can open the symbolic link to send I/O requests for the driver

I/O Control (IOCTL) code

- The code itself describes:
  - DeviceType
  - FunctionCode
  - TransferType
  - RequiredAccess

- IOCTL code is a 32 bit value that contains several fields:

(msdn.microsoft.com)



I/O transfer types

- Buffered I/O
  - The operating system creates a nonpaged system buffer, equal in size to the application's buffer. For write operations, the I/O manager copies user data into the system buffer before calling the driver stack. For read operations, the I/O manager copies data from the system buffer into the application's buffer after the driver stack completes the requested operation. (MSDN)
- Direct I/O
  - The operating system locks the application's buffer in memory. It then creates a memory descriptor list (MDL) that identifies the locked memory pages, and passes the MDL to the driver stack. Drivers access the locked pages through the MDL. (MSDN)
- Neither Buffered Nor Direct I/O
  - The operating system passes the application buffer's virtual starting address and size to the driver stack. The buffer is only accessible from drivers that execute in the application's thread context. (MSDN)

Communication between user mode program and kernel driver - 1

- CreateFile() to initialize access to device through its symbolic link

- Communication with
  - DeviceIoControl() // IOCTL call
  - WriteFile() // pass “stream” data
  - ReadFile() // receive “stream” data

Communication between user mode program and kernel driver - 2

- I/O Request through kernel32.DeviceIoControl

```
 BOOL WINAPI DeviceIoControl(
  _In_          HANDLE hDevice,
  _In_          DWORD dwIoControlCode,
  _In_opt_      LPVOID lpInBuffer,
  _In_          DWORD nInBufferSize,
  _Out_opt_     LPVOID lpOutBuffer,
  _In_          DWORD nOutBufferSize,
  _Out_opt_     LPDWORD lpBytesReturned,
  _Inout_opt_   LPOVERLAPPED lpOverlapped
);
```

Identify your target

- WinObj, DriverView
  - Gather useful information of loaded drivers
- IrpTracker
  - Monitor I/O requests
  - Detailed view on IRP

Analysis, bug hunting

- Read and understand the source code – if possible

- Reverse engineering
  - IDA

- Fuzzing
  - ioctlbf
  - ioctlfuzzer
  - Kartoffel
  - Your own scripts!

- Kernel debugging
  - Usually set the first breakpoint on function referenced by IRP_MJ_DEVICE_CONTROL

Why we love METHOD_NEITHER?

- No I/O Manager interaction on our buffer
  - Buffer pointers are given directly to driver
  - Try to point to kernel space... you may be able to overwrite code, return address, jump tables, function pointers, kernel structures with a simple IOCTL call

Write-What-Where

- Possibilities are not limited to METHOD_NEITHER output pointer manipulation

- Sometimes the WHAT is limited
  - In size
    (e.g. mov [eax], ecx)
  - In value
    (e.g. mov [eax], 1)

- Sometimes the value on the pointed memory increased or decreased with a constant number only
  (e.g. dec [eax])

- You still have a lot of opportunity to exploit these cases

Interesting targets to Write-What-Where

- Function pointers
- Jump tables
- System Service Dispatch table
- Interrupt Descriptor Table
- Global Descriptor Table
- Etc.

Code execution

- Our payload will run in kernel mode
  - User mode shellcodes will not work
  - It is possible to create fully kernel mode shellcode, but it is not so comfortable

- Build staged shellcodes/payloads
  - Elevate the attacker privileges in kernel mode
  - Return to the elevated user space process
  - Run user mode shellcode

Privilege escalation with token stealing - 1

- Find ETHREAD/KTHREAD (FS:[124h])

- ETHREAD → EPROCESS

- EPROCESS
  - UniqueProcessId (is it system process?)
  - ActiveProcessLinks (for the next EPROCESS structure)
  - Token (security descriptor of a process)

- Replace our EPROCESS Token pointer with the identified system process token pointer

Privilege escalation with token stealing - 2

```
nt!_KTHREAD
   +0x000 Header: _DISPATCHER_HEADER
   ...
   +0x040 ApcState: _KAPC_STATE
      +0x000 ApcListHead: [2] _LIST_ENTRY
      +0x010 Process: Ptr32 _KPROCESS
   ...
 nt!_EPROCESS
    +0x000 Pcb: _KPROCESS
    ...
    +0x0b4 UniqueProcessId: Ptr32 Void
    +0x0b8 ActiveProcessLinks: _LIST_ENTRY
       +0x000 Flink: Ptr32 _LIST_ENTRY
       +0x004 Blink: Ptr32 _LIST_ENTRY
    ...
    +0x0f8 Token: _EX_FAST_REF
       +0x000 Object: Ptr32 Void
       +0x000 RefCnt: Pos 0, 3 Bits
       +0x000 Value: Uint4B
```

Kernel debugging environment

- Two machines (debugger, debuggee)
  - Physical machine to physical machine
  - Virtualization
    - Physical machine to virtual machine
    - Virtual machine to virtual machine
    - Virtual serial port

Enable kernel debugging on target

- No more boot.ini in modern Windows systems
- Boot Configuration Data Editor: bcdedit
  - bcdedit /dbgsettings serial debugport:1 baudrate:115200
  - bcdedit /debug on

DEMO

Conclusion

- It was just an introduction, this is just tip of the iceberg...

- It is not magic

- There are tons of bugs in kernel drivers; you should focus on them

- You can do anything in kernel space
(also, easily crash the machine)

- Try to search and exploit driver vulnerabilities for fun... or for profit :)

Thank you for your attention!

Andras Kabai
contact (_at_) kabaiandras.hu

gr33t1ngz 4 @hekkcamp p4rt1c1p4nts!